OJT

OJTFlow

AI-Governed Healthcare Data Operations

Foundation Model | RAG | Persistent KG | MCP | Multi-Agent

Member: Nguyen Tran Nhat Trung

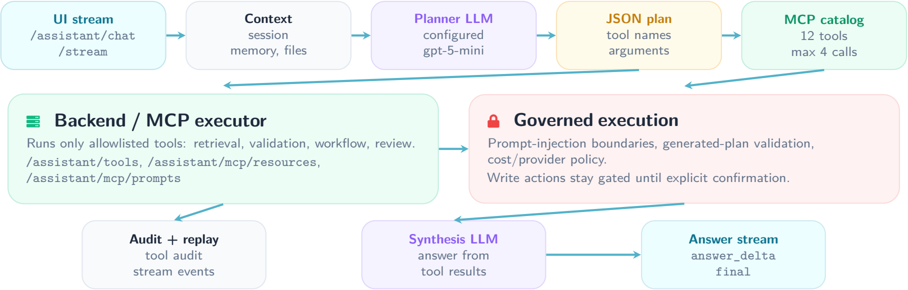
HuReDee-IoYou AI OJT Program | June 2026

Why OJTFlow?

Today	With OJTFlow
Manual, error-prone conversion	AI + rule-based execution
LLM hallucinations in output	RAG-grounded, validated output
No audit trail or approvals	Full audit + human review
PHI exposure risk	Auto-detect & redact PHI
Schema drift undetected	Schema registry + alerts

5 Agents | 35+ Endpoints | 6 MCP Servers | MarkItDown/MinerU | Persistent Knowledge Graph

LLM + MCP Runtime Flow



Live timeline: `stream_opened` → `planning_started` → `plan_ready` → `tool_started/completed` → `synthesis_started` → `answer_delta` → `final`

System Architecture

USER
- React 18 + Radix UI
- TypeScript
- AI Chat
- MCP Client

↓

ORCH
- FastAPI
- Workflow
- Review Gate
- MarkItDown / MinerU

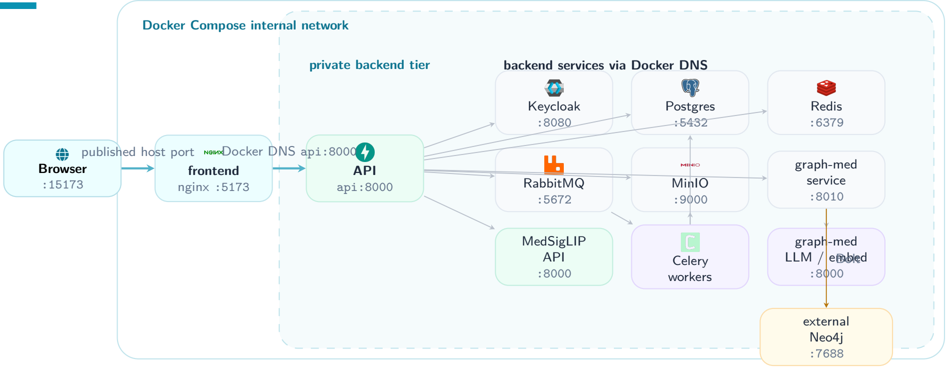
↓

CORE
- Contracts
- Policies
- Data Tools
- 5 Agents

↓

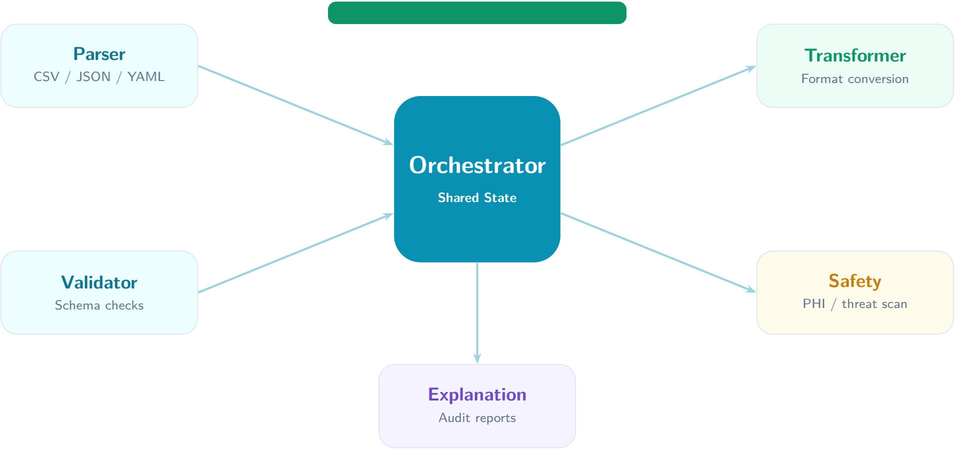
INFRA
- PostgreSQL
- Keycloak
- MinIO
- Redis
- Docker

Docker Internal Network

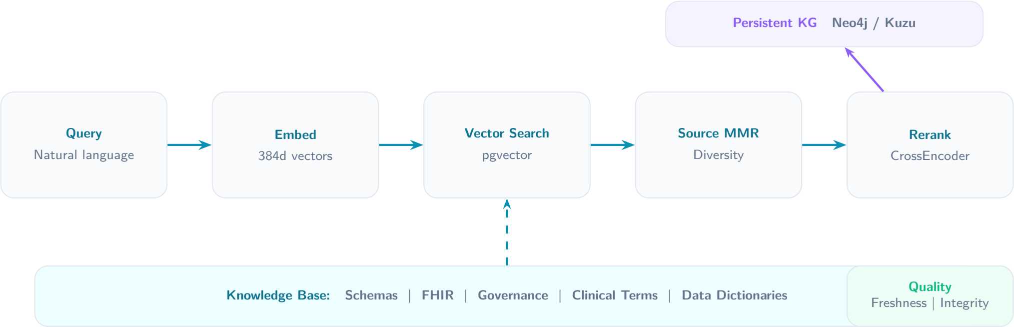


Only browser-facing ports are public; backend-to-backend calls stay inside Docker DNS. Cloud equivalent: HTTPS ingress + private VPC services.

Multi-Agent System



RAG Retrieval Pipeline



MedSigLIP Pretrain: ENTRep + Contrastive Learning

ENTRep Challenge
ACM MM 2025
ENT endoscopy images

→

Image-text pairs
ENT image + label
Clinical text prompt
Retrieval supervision

→

Contrastive pretrain
Image encoder
Text encoder
Shared embedding space

How it is used: ENTRep adapts OJTFlow MedSigLIP. Matched image-text pairs become close; wrong pairs become far. Used for classification, text-to-image search, and similar-image retrieval.

Source: aichallenge.hcmus.edu.vn/acm-mm-2025/entrep

Persistent Corpus Knowledge Graph

Corpus chunks
PDF, FHIR, CSV, notes

→

Coded registry
`medical_concepts.json`

→

KG Repository
Neo4j default
Kuzu / memory adapters

→

/knowledge UI
Graph + Search tabs

Deterministic import

Builder matches aliases, display names, and trusted codes; unresolved free text is not promoted to graph nodes.

Graph exploration API

`/search`, `/neighborhood`, `/nodes/{id}`
`/stats`, `/import` under `/api/v1/knowledge-graph`.

Workspace scope

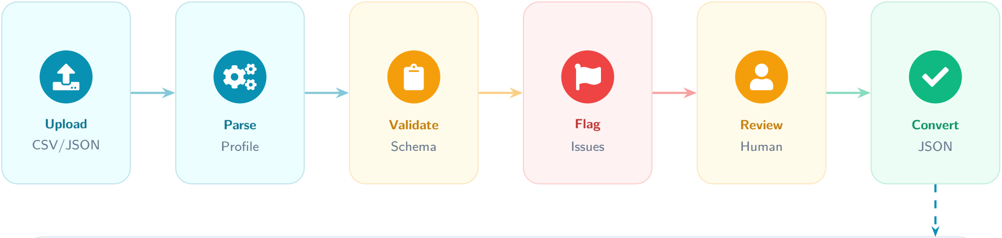
Reads merge global concepts with the caller organization; write paths keep organization provenance.

Evidence provenance

Nodes and edges carry chunk IDs, source snippets, confidence, and review state for traceable RAG.

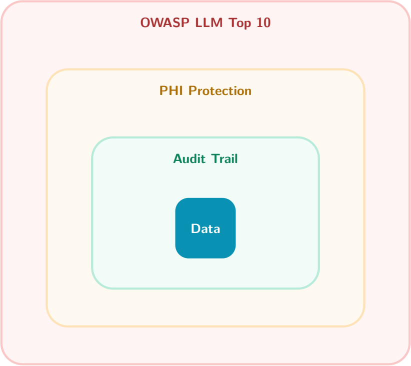
Graph-med-compatible Neo4j ontology + OJT source-backed overlay

End-to-End Data Flow



Output: Clean JSON + Explanation Report + Audit Events + Evidence Attachments

Security & Governance



- Prompt injection defense
- Auto-redact sensitive fields
- Hash-chain event integrity
- Keycloak SSO + HTTP-Only cookies
- Human gate for risky ops
- 4 deploy modes with enforced caps

Production Operations

Durable background jobs

RabbitMQ + Celery routes OCR, RAG, embedding, ingestion, and export worl

User-owned `/api/v1/jobs` lifecycle: list, inspect, cancel

Observability stack

Prometheus `/metrics`, Grafana dashboard, RabbitMQ/Celery exporters

Loki + Promtail logs and Sentry hooks for API/worker runtimes

Identity federation

Keycloak realm import with Google identity-provider configuration

HTTP-only cookies, service accounts, organization-scoped roles

Knowledge graph workspace

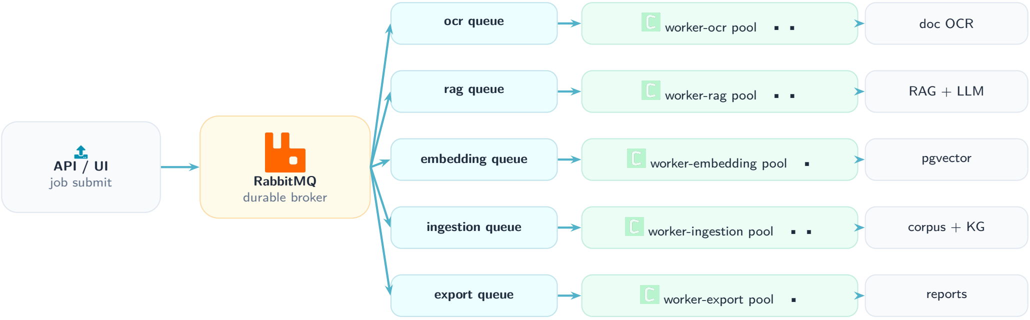
`/knowledge` Graph tab for concept search, neighborhoods, and provenance

Backed by `/api/v1/knowledge-graph` with Neo4j/Kuzu/memory adapters

worker-ocr · worker-rag · worker-embedding · worker-ingestion · worker-export

`make queue-stack` **starts RabbitMQ, workers, Flower, Prometheus, Grafana, Loki, and Promtail**

Job Queue Pool



Flower monitors tasks | Prometheus scrapes exporters | Grafana shows queue depth, latency, and failures

Document Extraction: MarkItDown + MinerU

PDF / images → DOCX / PPTX / XLSX → CSV / JSON / XML →

Extractor router
`auto`, `markitdown`, `mineru`

MarkItDown path

- Format converters for PDF, Office, HTML, CSV, JSON, XML, ZIP, audio, images
- Optional dependencies load only the needed parser
- `markitdown-ocr` plugin can run LLM Vision OCR for embedded/scanned images
- OJT gates OCR by size, provider policy, API key, and PHI controls

MinerU path

- `magic-pdf[full]` parses complex PDF/image layouts
- Removes headers, footers, page numbers and restores reading order
- Extracts tables, figures, formulas, captions, and scanned-page OCR
- Produces Markdown, JSON, and intermediate layout artifacts

ParserAgent result: Markdown text + extractor metadata + page count + warnings

parse/profile → validation → corpus chunks → RAG retrieval and persistent KG provenance

Technology Stack

Backend	Python 3.12 | FastAPI | PostgreSQL | Redis | MCP | MarkItDown | MinerU
Frontend	React 18 | TypeScript | Vite | Tailwind v4 | Radix UI
AI / ML	ENTRep Contrastive Pretrain | MedSigLIP | RAG | CrossEncoder | Neo4j / Kuzu
Infra	Docker | nginx | Keycloak | RabbitMQ | Celery | Prometheus | Grafana
Standards	FHIR | OWASP | OIDC / Keycloak | OCR

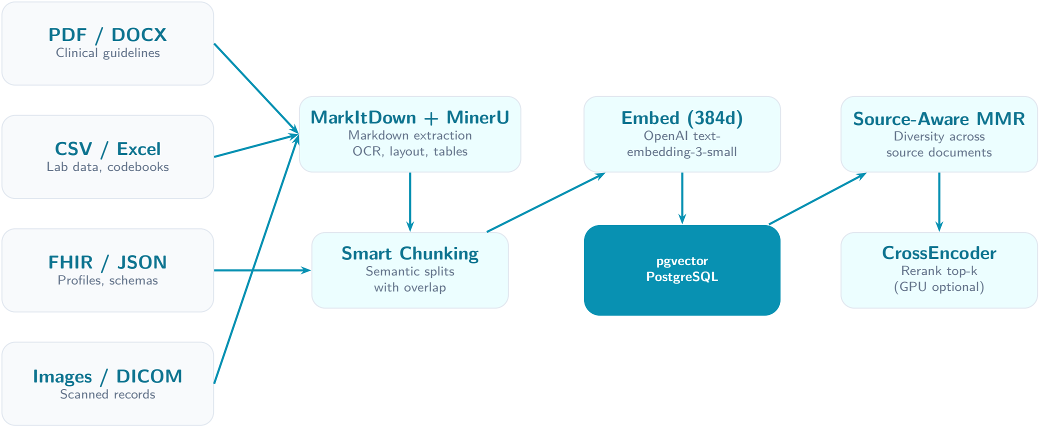
10 MODULES

5 AGENTS

35+ ENDPOINTS

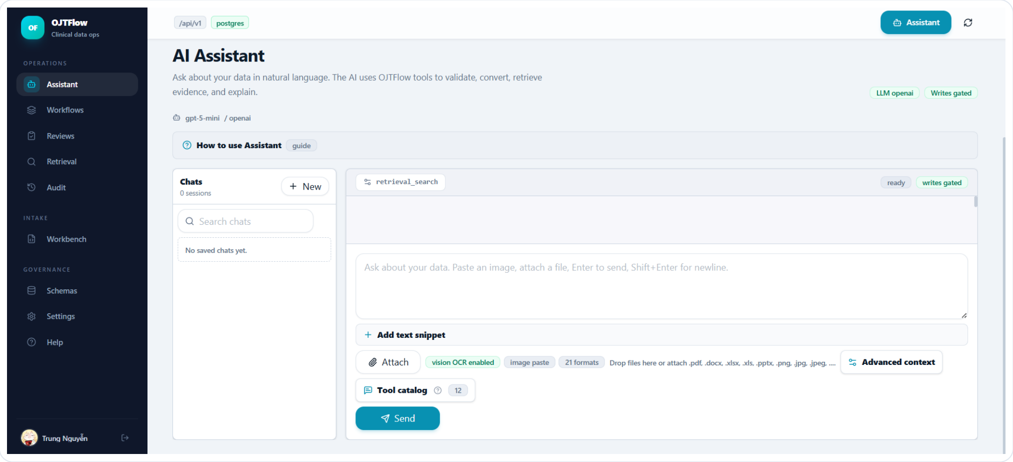
6 MCP SERVERS

Medical Knowledge Upload + Advanced RAG



MinIO Object Storage Original files preserved with versioning | Freshness gates | Integrity checksums | Source registry

AI Assistant Console Demo



Demo UI for retrieval, OCR upload, tool catalog access, and write-gated assistant actions.

OJT

Thank You

Foundation Model + RAG + Persistent KG + MCP + Multi-Agent

Governed Healthcare Data Operations

Questions? | HuReDee-IoYou AI OJT